Week 8 Report

Computational Statistics for Data Science · ISS507700

Student ID: 112006234

April 19, 2026

112006202 helped in discussing for narrow and flat dist. of 2(a)

I have largely solved the problems without the use of any AI agent or tool.

Q1 · Make an automated recommendation system for the PicCollage mobile app.

(a) Explore to see if any sticker bundles seem intuitively similar:

(i) Take a look at PicCollage's style and content of various bundles in their Sticker Store.

I used the Android version of PicCollage, which did not show recommendation lists for bundles.

However, I could still explore the PicCollage sticker store and observed that bundles are grouped by themes such as holidays, emotions, and aesthetics. Similar bundles tend to share visual style and purpose.

(ii) Find a single sticker bundle, then use intuition to recommend five other bundles in dataset.

I chose the bundle "autumn" because it is available in both the dataset and the PicCollage app. I predict that these are the 5 other recommended bundles: simplyautumn, fallinlovewiththefall,helloautumn, warmncozy, givethanks.

These bundles are similar because they all share seasonal theme (fall/autumn) and also have similar visual tone (warm colors).

(b) Find similar bundles using geometric models of similarity:

(i) Create cosine-similarity based recommendations for a given bundle

1. Create a cosine_sim(a, b) function that returns the cosine similarity of two vectors.

```
# load the PicCollage bundle data
ac_bundles <- read.csv("piccollage_accounts_bundles.csv", header = TRUE)

# remove the first column (account_id) and keep only bundle usage counts
items_matrix <- as.matrix(ac_bundles[, -1])

# cosine similarity between two vectors
cosine_sim <- function(a, b) {
  sum(a * b) / (sqrt(sum(a^2)) * sqrt(sum(b^2)))
}
```

The cosine similarity between two vectors is the dot product divided by the product of their magnitudes. A higher value means the two bundles have more similar usage patterns across accounts.

2. Create a cosine_recos(items_matrix, item_name) function that takes a matrix of items and the name of a specific item you are interested in, and returns five (5) recommendations for that item based on cosine similarity.

```
# return the top 5 most similar bundles to item_name
cosine_recos <- function(items_matrix, item_name) {
  # extract the column vector of the chosen bundle
  item_vector <- items_matrix[, item_name]

  # compute cosine similarity between the chosen bundle and every bundle
  sims <- apply(items_matrix, 2, function(x) cosine_sim(x, item_vector))

  # remove the bundle itself from the recommendation list
  sims <- sims[names(sims) != item_name]

  # sort from highest similarity to lowest
  sims <- sort(sims, decreasing = TRUE)

  # return the top 5 bundle names
  top_names <- names(sims)[1:5]
  return(top_names)
}
```

3. Top 5 cosine recommendations for "autumn".

```
chosen_bundle <- "autumn"

cosine_top5 <- cosine_recos(items_matrix, chosen_bundle)
cosine_top5
```

```
#> [1] "mmlm"       "julyfourth" "bestdaddy"  "justmytype" "gudetama"
```

(ii) Create correlation based recommendations:

```
# mean-center each bundle column
bundle_means <- apply(items_matrix, 2, mean)

# repeat the bundle means for each row
bundle_means_matrix <- t(replicate(nrow(items_matrix), bundle_means))

# column-mean-centered matrix
items_matrix_mc_col <- items_matrix - bundle_means_matrix

# reuse the same recommendation function
cor_top5 <- cosine_recos(items_matrix_mc_col, chosen_bundle)
cor_top5
```

```
#> [1] "mmlm"             "julyfourth"       "tropicalparadise" "bestdaddy"
#> [5] "justmytype"
```

For correlation-based similarity, I first mean-centered each bundle column, then reused the same cosine recos function. This treats similarity as agreement relative to each bundle's average usage, rather than raw usage counts.

(iii) Create an adjusted-cosine based recommendations.

```
# mean-center each account row
account_means <- apply(items_matrix, 1, mean)

# repeat the account means for each column
account_means_matrix <- matrix(account_means,
                               nrow = nrow(items_matrix),
                               ncol = ncol(items_matrix),
                               byrow = FALSE)
```

```
# row-mean-centered matrix
items_matrix_mc_row <- items_matrix - account_means_matrix

# reuse the same recommendation function
adjcos_top5 <- cosine_recos(items_matrix_mc_row, chosen_bundle)
adjcos_top5
```

```
#> [1] "tropicalparadise" "julyfourth"       "gudetama"         "X8bit2"
#> [5] "sweetmothersday"
```

For adjusted cosine, I mean-centered each account row before computing bundle similarity. This reduces the effect of users who generally use many stickers or very few stickers overall.

(c) What is the conceptual difference in cosine similarity, correlation, and adjusted cosine?

Cosine similarity compares the raw direction of two bundle vectors, so it focuses on whether two bundles are used by the same accounts in similar proportions. Correlation is cosine similarity after mean-centering each bundle, so it compares whether bundles rise and fall similarly relative to their own average usage. Adjusted cosine instead mean-centers each account, so it removes each user's overall sticker usage tendency and focuses more on preference differences between bundles. In short, cosine uses raw counts, correlation removes item-average effects, and adjusted cosine removes user-average effects.

A simpler way to think about it:

- Cosine: "Do these two bundles point in the same direction using raw counts?"
- Correlation: "Do these two bundles vary similarly relative to each bundle's own average?"
- Adjusted cosine: "Do users prefer these two bundles similarly after removing each user's personal baseline?"

(d) Apply the same cosine similarity idea to a very different dataset: the meaning of book descriptions as encoded in LLM embeddings.

(i) Pick a book that you are either familiar with or has an interesting description. Load the file description_embeddings.rds and use your cosine_sim() and cosine_recos() functions earlier to find the top 5 most similar books to a book of your choice.

```
# load book metadata and description embeddings
books <- read.csv("best_books_ever.csv", header = TRUE)
description_embeddings <- readRDS("description_embeddings.rds")
```

```
# check dimensions
dim(description_embeddings)
```

```
#> [1]   384 52477
```

```
chosen_book <- "Harry Potter and the Sorcerer's Stone"

# make sure the embedding column names match the book titles
# find the matching column
which(colnames(description_embeddings) == chosen_book)
```

```
#> [1] 33
```

```
# get top 5 similar books
book_top5 <- cosine_recos(description_embeddings, chosen_book)
book_top5
```

```
#> [1] "Harry Potter and the Prisoner of Azkaban"
#> [2] "Harry Potter and the Half-Blood Prince"
#> [3] "Harry Potter and the Goblet of Fire"
#> [4] "Blood Rites"
#> [5] "Backup"
```

(ii) Do the recommendations make sense? Why does cosine similarity "know" about similarity of book descriptions, without being told anything about plot or genre?.

Yes, the recommendations make sense.

The top results include other Harry Potter books such as Harry Potter and the Prisoner of Azkaban, Half-Blood Prince, and Goblet of Fire, which are clearly very similar in story, characters, and writing style. The presence of Blood Rites and Backup also makes sense because they are from similar fantasy genres.

Cosine similarity "knows" about similarity because the embedding vectors already encode the meaning of each book description. Even though we do not explicitly tell the model about genre or plot, the language model has transformed each description into a 384-dimensional vector where semantically similar texts are placed close together. Cosine similarity then compares the direction of these vectors, so books with similar themes, tone, and content end up with high similarity scores.

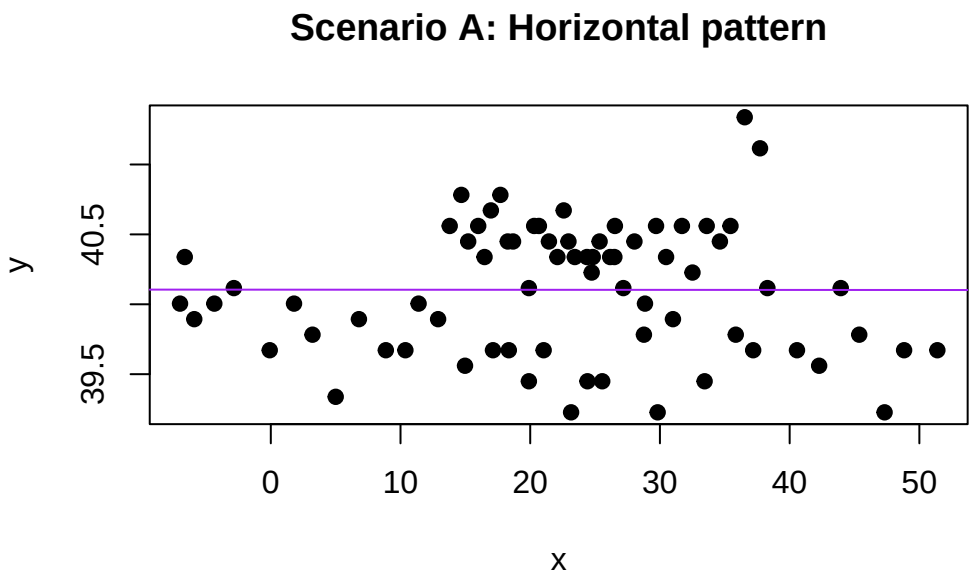
Q2 · Explore correlation further.

(a) Scenario A: Create a horizontal set of random points, with a relatively narrow but flat distribution.

```
library(compstatslib)
#pts_A <- interactive_regression()
#saveRDS(pts_A, "pts_A.rds")
pts_A <- readRDS("pts_A.rds")

plot(pts_A$x, pts_A$y, pch = 19,
     main = "Scenario A: Horizontal pattern",
     xlab = "x", ylab = "y")

abline(lm(pts_A$y ~ pts_A$x), col = "purple")
```




```
cor(pts_A)
```

```
#>              x           y
#> x  1.000000000 -0.001316268
#> y -0.001316268  1.000000000
```

```r
summary(lm(pts_A$y ~ pts_A$x))
```

```
#>
#> Call:
#> lm(formula = pts_A$y ~ pts_A$x)
#>
#> Residuals:
#>     Min      1Q  Median      3Q     Max
#> -0.8766 -0.4310  0.0128  0.3457  1.2352
#>
#> Coefficients:
#>               Estimate Std. Error t value Pr(>|t|)
#> (Intercept)  4.010e+01  1.082e-01 370.540   <2e-16 ***
#> pts_A$x     -4.516e-05  4.101e-03  -0.011    0.991
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 0.4656 on 70 degrees of freedom
#> Multiple R-squared:  1.733e-06,  Adjusted R-squared:  -0.01428
#> F-statistic: 0.0001213 on 1 and 70 DF,  p-value: 0.9912
```

The distribution is narrow because the y-values are tightly clustered around a constant value (around 40), meaning there is low variance in y. The points do not spread much vertically, even though x varies widely.

(i) What raw slope of x and y would you generally expect?

The slope is approximately 0 because y does not change much as x increases.

(ii) What is the correlation of x and y that you would generally expect?

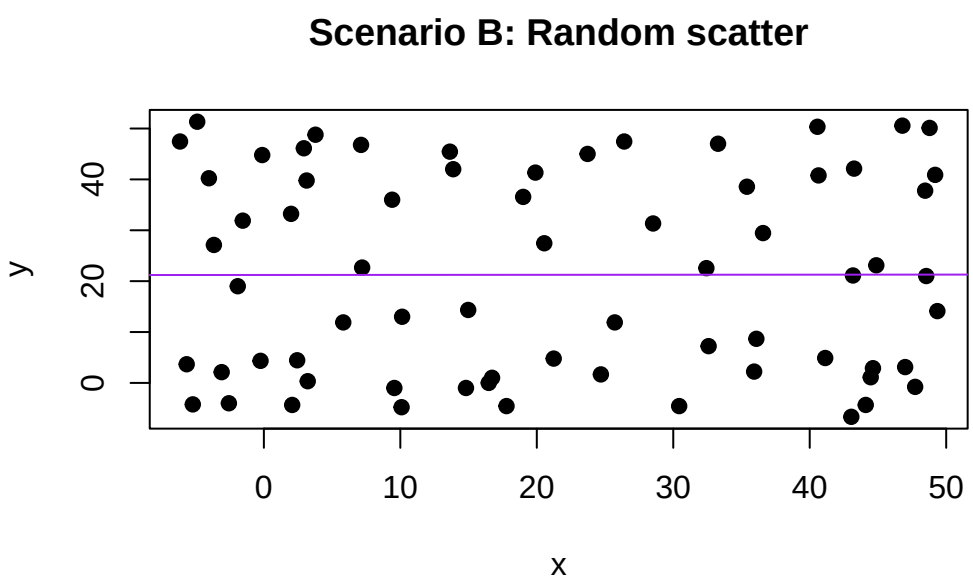
The correlation is close to 0 because there is no linear relationship between x and y.

(b) Scenario B: Create a random set of points to fill the entire plotting area, along both x-axis and y-axis.

```r
#pts_B <- interactive_regression()
#saveRDS(pts_B, "pts_B.rds")
pts_B <- readRDS("pts_B.rds")
```

```r
plot(pts_B$x, pts_B$y, pch = 19,
     main = "Scenario B: Random scatter",
     xlab = "x", ylab = "y")

abline(lm(pts_B$y ~ pts_B$x), col = "purple")
```




```r
cor(pts_B)
```

```
#>            x          y
#> x 1.00000000 0.00136688
#> y 0.00136688 1.00000000
```

```r
summary(lm(pts_B$y ~ pts_B$x))
```

```
#>
#> Call:
#> lm(formula = pts_B$y ~ pts_B$x)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -27.9407 -19.0550  -0.2219  19.5331  30.1301
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept) 21.215318   3.670828   5.779 2.22e-07 ***
#> pts_B$x      0.001448   0.130423   0.011    0.991
```

```
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 19.83 on 66 degrees of freedom
#> Multiple R-squared:  1.868e-06,  Adjusted R-squared:  -0.01515
#> F-statistic: 0.0001233 on 1 and 66 DF,  p-value: 0.9912
```

(i) What raw slope of x and y would you generally expect?

The slope is approximately 0 because the points are randomly distributed with no clear trend.

(ii) What is the correlation of x and y that you would generally expect?

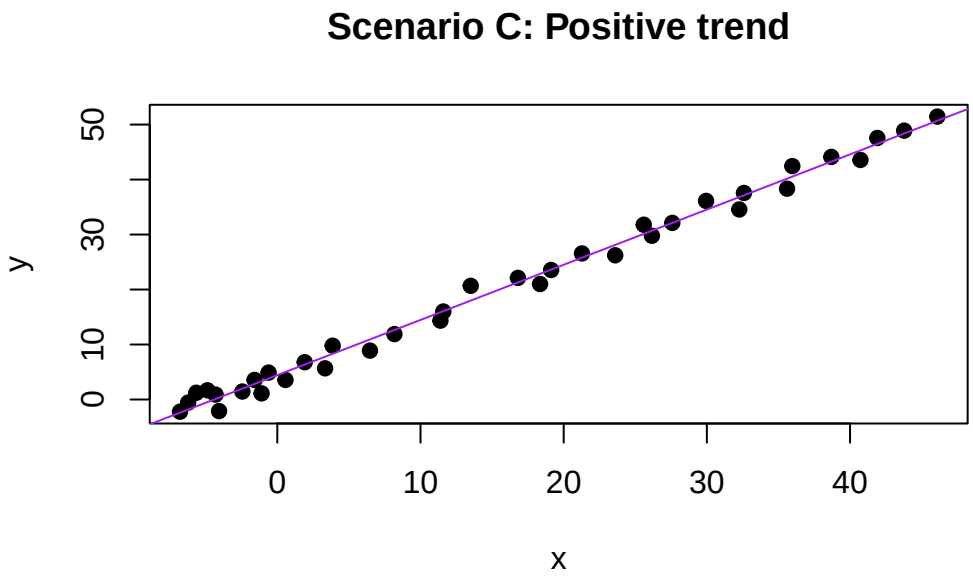
The correlation is close to 0 since there is no consistent linear pattern.

(c) Scenario C: Create a diagonal set of random points trending upwards at 45 degrees.

```r
#pts_C <- interactive_regression()
#saveRDS(pts_C, "pts_C.rds")
pts_C <- readRDS("pts_C.rds")

plot(pts_C$x, pts_C$y, pch = 19,
     main = "Scenario C: Positive trend",
     xlab = "x", ylab = "y")

abline(lm(pts_C$y ~ pts_C$x), col = "purple")
```




```
cor(pts_C)
```

```
#>           x         y
#> x 1.0000000 0.9961039
#> y 0.9961039 1.0000000
```

```
summary(lm(pts_C$y ~ pts_C$x))
```

```
#>
#> Call:
#> lm(formula = pts_C$y ~ pts_C$x)
#>
#> Residuals:
#>     Min      1Q  Median      3Q     Max
#> -2.4540 -1.5271  0.4199  1.0686  2.6829
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept)  4.43026    0.34375   12.89 7.54e-15 ***
#> pts_C$x      1.00367    0.01502   66.82  < 2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 1.525 on 35 degrees of freedom
#> Multiple R-squared:  0.9922, Adjusted R-squared:  0.992
#> F-statistic:  4465 on 1 and 35 DF,  p-value: < 2.2e-16
```

(i) What raw slope of x and y would you generally expect?

The slope is approximately 1 because y increases at the same rate as x.

(ii) What is the correlation of x and y that you would generally expect?

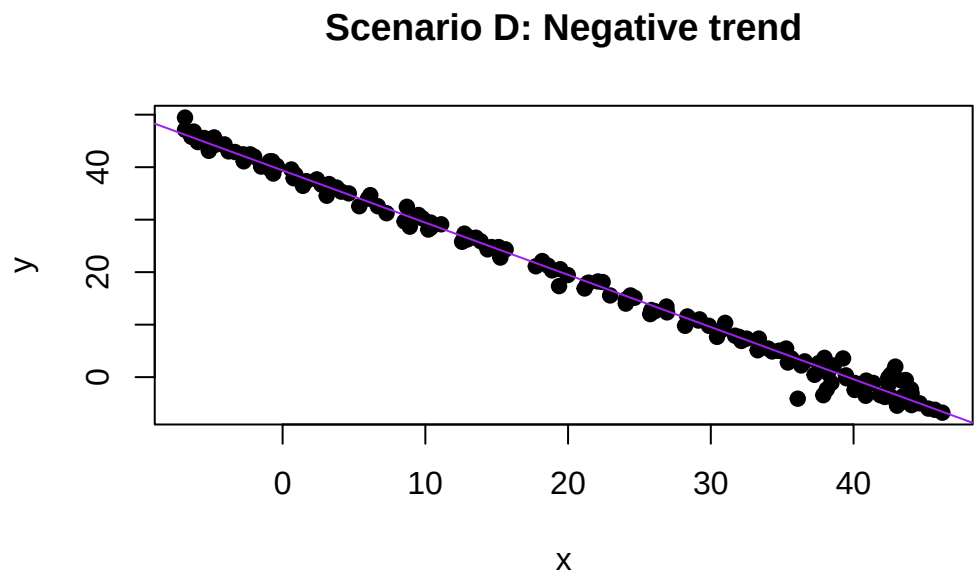
The correlation is close to 1 because there is a strong positive linear relationship.

(d) Scenario D: Create a diagonal set of random trending downwards at 45 degrees.

```
#pts_D <- interactive_regression()
#saveRDS(pts_D, "pts_D.rds")
pts_D <- readRDS("pts_D.rds")

plot(pts_D$x, pts_D$y, pch = 19,
     main = "Scenario D: Negative trend",
     xlab = "x", ylab = "y")

abline(lm(pts_D$y ~ pts_D$x), col = "purple")
```




```
cor(pts_D)
```

```
#>            x          y
#> x  1.0000000 -0.9960813
#> y -0.9960813  1.0000000
```

```
summary(lm(pts_D$y ~ pts_D$x))
```

```
#>
#> Call:
#> lm(formula = pts_D$y ~ pts_D$x)
#>
#> Residuals:
#>     Min      1Q  Median      3Q     Max
#> -7.5914 -0.7592  0.0489  0.6498  5.3187
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept) 39.380154   0.197351   199.5   <2e-16 ***
#> pts_D$x     -0.994639   0.007235  -137.5   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 1.509 on 149 degrees of freedom
#> Multiple R-squared:  0.9922, Adjusted R-squared:  0.9921
#> F-statistic: 1.89e+04 on 1 and 149 DF,  p-value: < 2.2e-16
```

(i) What raw slope of x and y would you generally expect?

The slope is approximately -1 because y decreases as x increases.

(ii) What is the correlation of x and y that you would generally expect?

The correlation is close to -1 because there is a strong negative linear relationship.

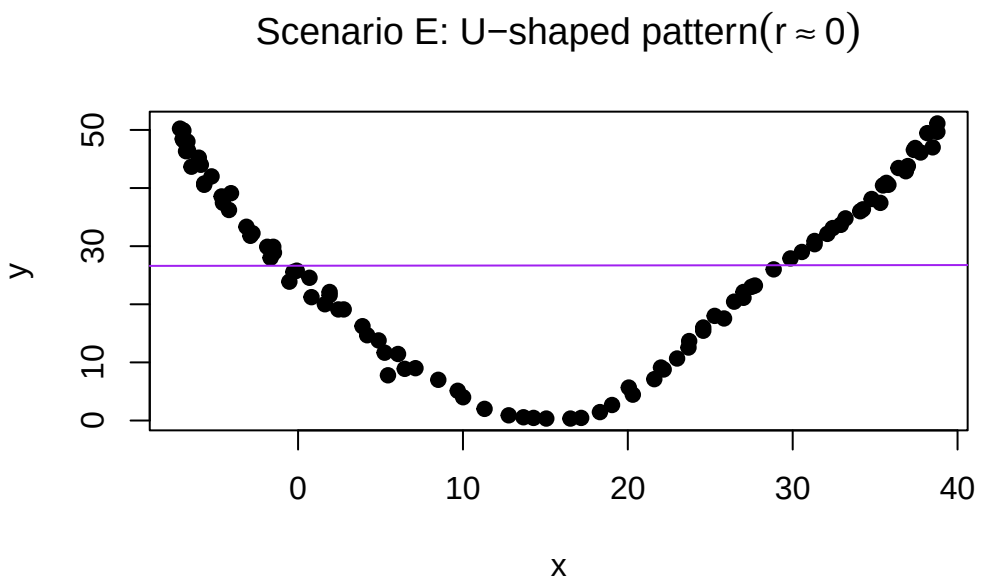
(e) A pattern that visually suggests a strong relationship but produces $r \approx 0$.

```
#pts_E <- interactive_regression()
#saveRDS(pts_E, "pts_E.rds")
pts_E <- readRDS("pts_E.rds")

plot(pts_E$x, pts_E$y, pch = 19,
     main = expression("Scenario E: U-shaped pattern" *  (r %~~% 0)),
```

```
     xlab = "x", ylab = "y")

abline(lm(pts_E$y ~ pts_E$x), col = "purple")
```




```
cor(pts_E)
```

```
#>             x          y
#> x 1.00000000 0.00291527
#> y 0.00291527 1.00000000
```

```
summary(lm(pts_E$y ~ pts_E$x))
```

```
#>
#> Call:
#> lm(formula = pts_E$y ~ pts_E$x)
#>
#> Residuals:
#>     Min      1Q  Median      3Q     Max
#> -26.346 -12.991   1.273  13.912  24.370
#>
#> Coefficients:
#>              Estimate Std. Error t value Pr(>|t|)
#> (Intercept) 26.635504   2.148616  12.397   <2e-16 ***
#> pts_E$x      0.002857   0.100012   0.029    0.977
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

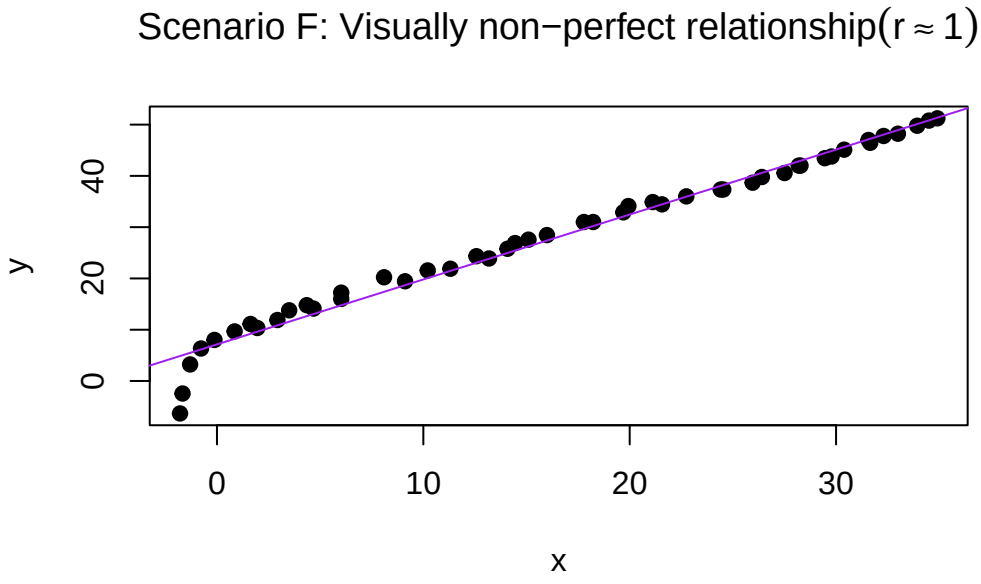
```
#>
#> Residual standard error: 15.75 on 96 degrees of freedom
#> Multiple R-squared:  8.499e-06,  Adjusted R-squared:  -0.01041
#> F-statistic: 0.0008159 on 1 and 96 DF,  p-value: 0.9773
```

(f) A pattern of data points with perfect correlation ($r \approx 1$), but visually suggests a non-perfect relationship.

```
#pts_F <- interactive_regression()
#saveRDS(pts_F, "pts_F.rds")
pts_F <- readRDS("pts_F.rds")

plot(pts_F$x, pts_F$y, pch = 19,
     main = expression("Scenario F: Visually non-perfect relationship" *  (r %~~% 1)),
     xlab = "x", ylab = "y")

abline(lm(pts_F$y ~ pts_F$x), col = "purple")
```




```
cor(pts_F)
```

```
#>           x         y
#> x 1.0000000 0.9884779
#> y 0.9884779 1.0000000
```

```r
summary(lm(pts_F$y ~ pts_F$x))
```

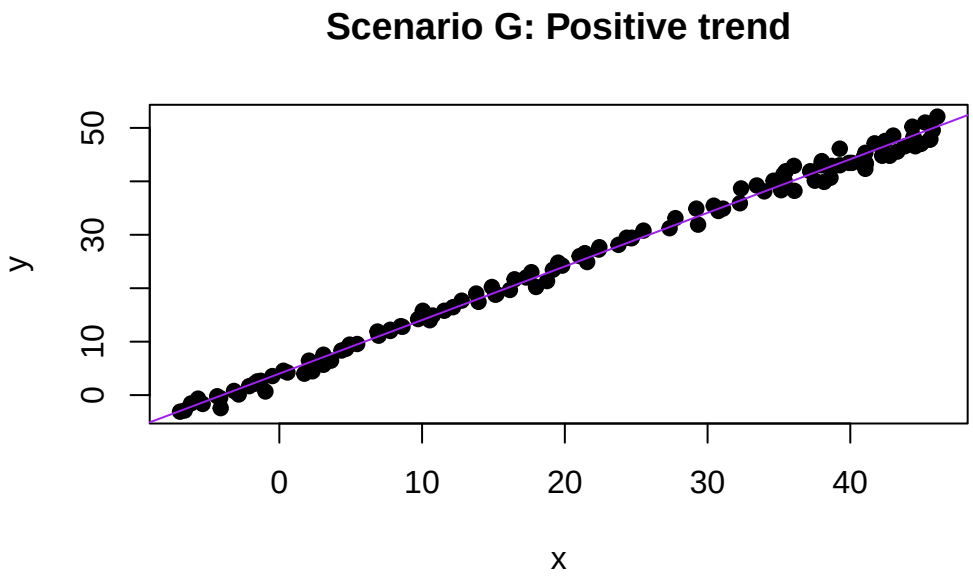
```
#>
#> Call:
#> lm(formula = pts_F$y ~ pts_F$x)
#>
#> Residuals:
#>      Min       1Q   Median       3Q      Max
#> -11.1990  -0.7291   0.3062   1.2514   2.8364
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept)  7.13636    0.58225   12.26  4.3e-16 ***
#> pts_F$x      1.26639    0.02859   44.29  < 2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 2.328 on 46 degrees of freedom
#> Multiple R-squared:  0.9771, Adjusted R-squared:  0.9766
#> F-statistic:  1962 on 1 and 46 DF,  p-value: < 2.2e-16
```

(g) How correlation relates to simple regression:

(i) Run the simulation and record the points you create:

```r
#pts_G <- interactive_regression()
#saveRDS(pts_G, "pts_G.rds")
pts_G <- readRDS("pts_G.rds")
plot(pts_G$x, pts_G$y, pch = 19,
     main = "Scenario G: Positive trend",
     xlab = "x", ylab = "y")

abline(lm(pts_G$y ~ pts_G$x), col = "purple")
```



(ii) Use the lm() function to estimate the regression intercept and slope of pts to ensure they are the same as the values reported in the simulation plot:

```
summary(lm(pts_G$y ~ pts_G$x))
```

```
#>
#> Call:
#> lm(formula = pts_G$y ~ pts_G$x)
#>
#> Residuals:
#>     Min      1Q  Median      3Q     Max
#> -2.8597 -0.5995  0.1003  0.9079  2.7164
#>
#> Coefficients:
#>             Estimate Std. Error t value Pr(>|t|)
#> (Intercept)  4.01361    0.18852   21.29   <2e-16 ***
#> pts_G$x      1.00318    0.00692  144.97   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 1.236 on 112 degrees of freedom
#> Multiple R-squared:  0.9947, Adjusted R-squared:  0.9947
#> F-statistic: 2.102e+04 on 1 and 112 DF,  p-value: < 2.2e-16
```

(iii) Calculate the correlation of x and y using cor(pts) — is it the same as reported in the plot?

```
cor(pts_G)
```

```
#>           x         y
#> x 1.0000000 0.9973459
#> y 0.9973459 1.0000000
```

The regression output matches the values shown in the simulation, and the correlation computed using cor is also consistent with the plot.

(iv) Standardize and re-estimate slope.

```
x_z <- scale(pts_G$x)
y_z <- scale(pts_G$y)

summary(lm(y_z ~ x_z))
```

```
#>
#> Call:
#> lm(formula = y_z ~ x_z)
#>
#> Residuals:
#>       Min        1Q    Median        3Q       Max
#> -0.169186 -0.035469  0.005932  0.053716  0.160708
#>
#> Coefficients:
#>               Estimate Std. Error t value Pr(>|t|)
#> (Intercept) -4.132e-17  6.850e-03       0        1
#> x_z          9.973e-01  6.880e-03     145   <2e-16 ***
#> ---
#> Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
#>
#> Residual standard error: 0.07313 on 112 degrees of freedom
#> Multiple R-squared:  0.9947, Adjusted R-squared:  0.9947
#> F-statistic: 2.102e+04 on 1 and 112 DF,  p-value: < 2.2e-16
```

(v) What is the relationship between correlation and the standardized simple-regression estimates?

After standardizing both x and y, the regression slope becomes equal to the correlation coefficient. This shows that correlation represents the standardized slope in simple linear regression and captures the strength and direction of the linear relationship independent of the original units.